TP2 – Reconstrucción 3D por estéreo multivista

I308 Visión Artificial – Universidad de San Andrés

Franco Needleman Lautaro García Conejero Luca Fassio

Octubre 2026

Resumen

Reconstruimos la nube de puntos 3D métrica de un caracol marino a partir de 83 pares estéreo tomados en órbita con una cámara ELP-USB3D1080P02-H120, usando un tablero ChArUco como marco de referencia global. El pipeline calibra y rectifica el par estéreo, calcula disparidad con SGBM, estima la pose de cada toma con `solvePnP`, fusiona las nubes en el marco del tablero y las optimiza con Open3D. Estimamos un tamaño de $122 \times 219 \times 202$ mm (alto × largo × ancho), frente a . . . medidos con regla.

1. Introducción y marco teórico

Modelo pinhole y distorsión. Un punto $\mathbf{X}_c = (X, Y, Z)$ en coordenadas de cámara se proyecta en $u = f_x X/Z + c_x$, $v = f_y Y/Z + c_y$, o en forma matricial $\lambda\,\tilde{\mathbf{u}} = K[R\,|\,\mathbf{t}]\tilde{\mathbf{X}}$ con $K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$. Los *parámetros intrínsecos* son las distancias focales en píxeles (f_x, f_y) y el punto principal (c_x, c_y). Las lentes reales agregan distorsión radial (k_1, k_2, k_3) y tangencial (p_1, p_2), que curvan las rectas. Hay que calibrar porque estos parámetros dependen de cada cámara y sin ellos no se puede relacionar un píxel con un rayo 3D, ni medir en milímetros.

Calibración estéreo. Además de los intrínsecos de cada cámara, se estiman los *extrínsecos relativos* $(R, \mathbf{t})$ que llevan puntos del sistema de la cámara izquierda al de la derecha: $\mathbf{X}_r = R\mathbf{X}_l + \mathbf{t}$. La norma $\|\mathbf{t}\|$ es la *baseline* y fija la escala métrica de la triangulación. Se estiman observando un patrón de geometría conocida (checkerboard) en muchas poses: para cada imagen se detectan las esquinas, y se minimiza por Levenberg–Marquardt la suma de los cuadrados de las distancias entre las esquinas detectadas y las proyectadas por el modelo, de forma conjunta sobre K_l, K_r, distorsiones, $(R, \mathbf{t})$ y la pose del patrón en cada vista (`cv.stereoCalibrate`). El *error de reproyección* RMS es la raíz del promedio de esas distancias en píxeles y mide qué tan bien explica el modelo las observaciones.

Geometría epipolar y rectificación. Dado un punto en la imagen izquierda, su correspondiente en la derecha está sobre una recta, la *línea epipolar* ($\tilde{\mathbf{u}}_r^\top F \tilde{\mathbf{u}}_l = 0$). La rectificación aplica a cada imagen una homografía (rotaciones R_1, R_2 y nuevas proyecciones P_1, P_2, `cv.stereoRectify`) y quita la distorsión, de modo que ambos planos imagen quedan coplanares y las líneas epipolares son horizontales y en la misma fila. Así la búsqueda de correspondencias pasa de 2D a 1D: para cada píxel solo se busca a lo largo de la misma fila, lo que hace viable y robusto el cálculo denso de disparidad.

De disparidad a profundidad. En un par rectificado, con focal f y baseline B, un punto a profundidad Z aparece en columnas x_l y x_r con disparidad $d = x_l - x_r$ y, por triángulos semejantes, $Z = fB/d$. La profundidad es inversamente proporcional a la disparidad, así que el error crece como $\delta Z \approx Z^2\,\delta d/(fB)$. La matriz Q de `stereoRectify` expresa esta relación en forma homogénea: $[X\ Y\ Z\ W]^\top = Q[x\ y\ d\ 1]^\top$ (`cv.reprojectImageTo3D`).

2. Metodología

Dataset propio. Usamos la cámara ELP de la cátedra (dos sensores de 1920×1080, baseline nominal de 60 mm). Tomamos 52 pares de calibración con un checkerboard de 9×6 esquinas internas y cuadros de 28 mm, y 83 pares del objeto en una órbita completa sobre un ChArUco de 9×13 cuadros de 42 mm (marcadores de 31.5 mm, `DICT_5X5_100`), que es el marco global (x, y sobre el tablero, z hacia la mesa).

Calibración. Implementada con funciones de OpenCV (`src/calibration.py`). Las esquinas se detectan con `findChessboardCorners` sobre la imagen reducida al 70 % y se refinan con `cornerSubPix` a resolución completa. Se calibra con `stereoCalibrateExtended`, que devuelve el error por vista, y se descartan iterativamente los pares con error atípico (umbral: mediana $+\,2{,}5 \cdot 1{,}4826{\cdot}$MAD, con piso de 1 px); se usaron 32 pares. Como la órbita de captura cubre distancias y ángulos distintos a los de la calibración, refinamos $(R, \mathbf{t})$ con 12 vistas del ChArUco (capturas 0, 6, . . . , 66) fijando los intrínsecos (`CALIB_FIX_INTRINSIC`). Validamos este paso en los 71 pares restantes. Los mapas de rectificación se calculan con `stereoRectify` ($\alpha = 0$) e `initUndistortRectifyMap`.

Disparidad. `StereoSGBM` (`src/disparity.py`) sobre las imágenes rectificadas en grises: `numDisparities`=192 (cubre de ∼190 mm al infinito, ya que $Z = fB/d \approx 600 \cdot 59/192$), `blockSize`=7, $P_1 = 8 \cdot 7^2$, $P_2 = 32 \cdot 7^2$ (suavidad estándar de OpenCV), `uniquenessRatio`=5, `speckleWindowSize`=100, `speckleRange`=2, `disp12MaxDiff`=1

(chequeo izquierda-derecha). Los valores se ajustaron visualmente: bloques más chicos generaban ruido en el tablero blanco, y una unicidad mayor vaciaba las zonas de poca textura.

Nube local y pose global. La nube local se obtiene con `reprojectImageTo3D` y Q, conservando los píxeles con $d > 0$ y $Z > 0$, y el color de la imagen izquierda. Está expresada en la cámara izquierda *rectificada*. La pose se estima detectando el ChArUco en la imagen izquierda *original* (`CharucoDetector`) y resolviendo `solvePnP` con K_l y la distorsión izquierda. Se rechazan las tomas con menos de 8 esquinas, esquinas colineales, cobertura menor al 2 % de la imagen o RMS de reproyección mayor a 5 px. `solvePnP` devuelve $(\mathbf{r}, \mathbf{t})$ tales que $\mathbf{X}_c = R\,\mathbf{X}_w + \mathbf{t}$, donde $\mathbf{r}$ es el *vector de Rodrigues*: su dirección es el eje de giro y su norma el ángulo en radianes. `cv.Rodrigues` lo convierte en la matriz R. La transformación de la nube al mundo es

$$\mathbf{X}_w = \underbrace{\begin{bmatrix} R^\top & -R^\top \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{T_{w \leftarrow c}} \underbrace{\begin{bmatrix} R_1^\top & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{T_{c \leftarrow \mathrm{rect}}} \mathbf{X}_{\mathrm{rect}},$$

donde $R_1^\top$ deshace la rotación de rectificación y $T_{w \leftarrow c}$ invierte la pose de PnP.

Fusión y optimización (`src/fusion.py`). Las capturas se procesan de a una, sin acumular nunca las nubes completas (∼1.4 M puntos cada una):

1. *Corrección por plano del tablero.* La profundidad estéreo y la pose PnP difieren unos milímetros por vista, así que el tablero reconstruido no cae exactamente en $z = 0$. Se ajusta un plano con RANSAC (`segment_plane`, umbral de 2 mm) a los puntos cercanos al tablero, y se aplica la rotación y traslación mínimas que lo llevan a $z = 0$.
2. *Filtrado espacial*: se seleccionan los puntos dentro de una `OrientedBoundingBox` alineada al tablero (`get_point_indices_within_bounding_box` y `select_by_index`), con $x \in [40, 290]$, $y \in [140, 410]$ y $z \in [-160, -2]$ mm. El límite $z < -2$ mm excluye la superficie del tablero.
3. *ICP punto a plano* (opcional) de cada captura contra una primera fusión limpia (distancia máxima de 3 mm).
4. *Consistencia multivista*: se conservan los voxels de 3 mm observados por al menos 20 capturas. Los *flying pixels* que SGBM genera en los bordes de profundidad (interpolando entre el objeto y el tablero) y los fragmentos flotantes dependen del punto de vista, por lo que pocas capturas los respaldan. La superficie real aparece en decenas.
5. *Submuestreo* con `voxel_down_sample` de 1 mm: conserva el detalle de las espinas del caracol y reduce la redundancia entre vistas.
6. *Outliers*: `remove_statistical_outlier` (30 vecinos, 2σ) y luego el cluster DBSCAN más grande ($\varepsilon = 5$ mm).

Por último se estiman normales (radio de 4 mm, orientadas hacia afuera), se exporta `caracol.ply` en binario y se genera una malla de Poisson (profundidad 9) como opcional.

Medición métrica. El alto es la distancia al plano $z = 0$ del punto más alto (percentil 99.5 de $-z$). El largo y el ancho son los lados del rectángulo de área mínima (`cv.minAreaRect`) que encierra la proyección de la nube sobre el tablero, lo que equivale a una *bounding box* orientada con un eje vertical. El resultado es métrico, y no "salvo escala" como en la estructura a partir del movimiento monocular, por dos razones: la baseline B se calibró con un patrón de tamaño conocido, lo que fija $Z = fB/d$ en milímetros, y el marco global está definido por un tablero de dimensiones conocidas.

3. Resultados y discusión

3.1. Calibración y rectificación

	Cámara izquierda	Cámara derecha
f_x, f_y (px)	600.97, 598.79	603.37, 601.50
c_x, c_y (px)	961.81, 532.43	956.05, 539.48
k_1, k_2, p_1, p_2, k_3	0,0077, −0,0258, 0,0004, −0,0013, 0,0041	0,0067, −0,0247, 0,0003, −0,0017, 0,0039
$\mathbf{t} = (-59{,}22,\ -0{,}31,\ 0{,}89)$ mm (baseline 59.23 mm, nominal 60 mm); R: rotación de 0,63° (casi paralelas)		
RMS de reproyección: 0.642 px (checkerboard, 32 pares)	1.475 px (refinamiento R/T, 12 vistas ChArUco)	

Cuadro 1: Parámetros de calibración obtenidos (imágenes de 1920 × 1080).

El RMS de 0.64 px con el checkerboard indica una calibración buena (subpíxel). El RMS del refinamiento es mayor porque fija los intrínsecos y usa vistas oblicuas y lejanas del ChArUco, así que no es comparable. Para comparar las

dos calibraciones medimos sobre 66 pares de captura que no se usaron en el refinamiento: *(i)* el error vertical $|\Delta y|$ entre esquinas correspondientes después de rectificar, y *(ii)* la escala de las distancias entre esquinas trianguladas respecto de las reales (Tabla 2). El refinamiento reduce el error epipolar un 30 % y elimina un sesgo de escala del −1,85 %, que habría encogido el objeto unos 4 mm. La Figura 1 muestra que las filas quedan alineadas después de rectificar.

Calibración	$\|\Delta y\|$ medio (px)	Escala media	Desvío de escala entre vistas
Solo checkerboard	0.738	0.9815	0.0165
Refinada con ChArUco	**0.517**	**1.0002**	0.0158

Cuadro 2: Validación en 66 pares no usados para calibrar.

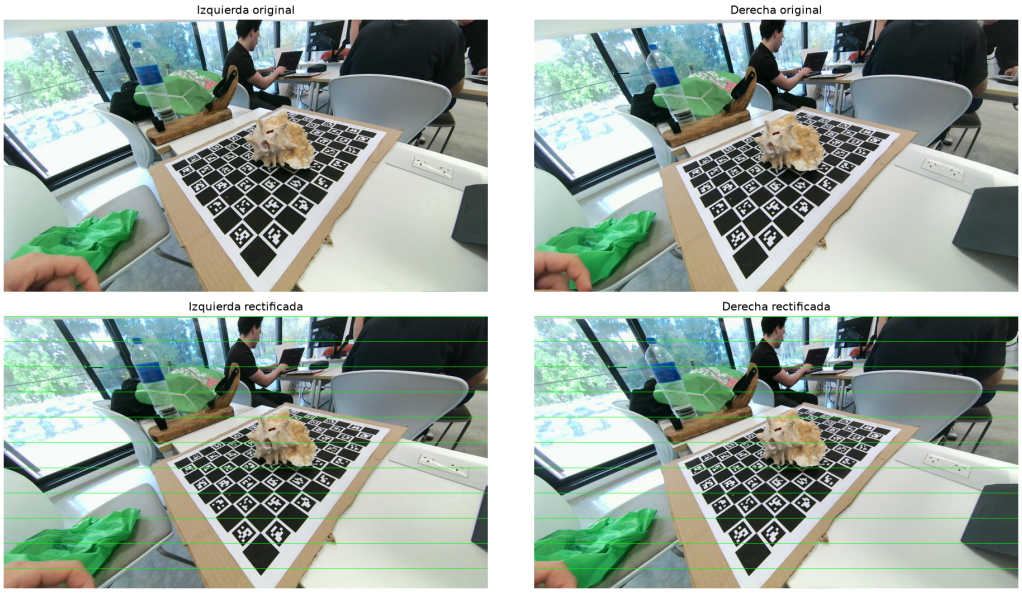
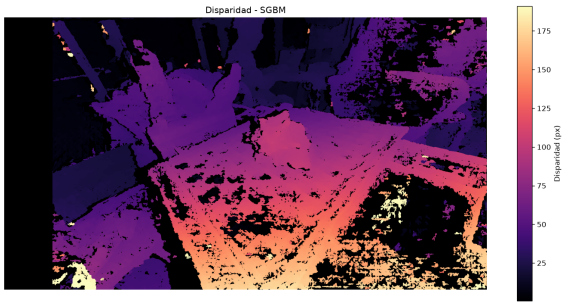


Figura 1: Par original (arriba) y rectificado (abajo, con líneas horizontales cada 100 px), captura 1.

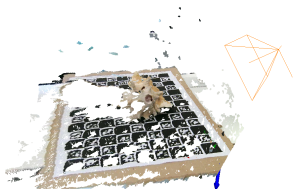
3.2. Disparidad, nube local y poses



(a) Disparidad SGBM.



(b) Nube local (cámara rectificada).



(c) Nube en el marco del tablero.

Figura 2: Captura 1. En (c) se ven los ejes del mundo (esquina del ChArUco, z en azul hacia la mesa) y el frustum de la cámara.

La Figura 2 muestra un mapa de disparidad representativo: el 67 % de los píxeles tiene disparidad válida (entre 42 % y 79 % según la captura). El caracol queda en $d \approx 100$–125 px ($Z \approx 280$–350 mm). Los huecos se concentran en los cuadros negros y en la mesa blanca, donde no hay textura, y en las 192 columnas de la izquierda, que no tienen correspondencia posible. En la parte inferior, la más cercana a la cámara, la disparidad se satura cerca de 192 px: ahí `numDisparities` se queda corto, pero esa zona cae fuera del objeto. La nube local de cada par tiene entre 0.9 y 1.6 M de puntos.

Se aceptaron 77 de 83 capturas. La 33 se rechazó porque el tablero ocupa el 1.5 % de la imagen, y las 71, 74–76 y 78 porque detectan menos de 8 esquinas (el objeto o el encuadre ocultan el tablero). El RMS de reproyección de

`solvePnP` es de 1.05 px en mediana (0.41–2.86 px). La Figura 3 muestra las poses estimadas: las cámaras rodean el tablero por completo, a alturas de 35 a 590 mm y a 100–475 mm del centro, con los ejes ópticos apuntando hacia el objeto. Eso coincide con la órbita a mano alzada, de alturas variables, con la que se tomaron las capturas, y la numeración avanza alrededor del tablero en el orden en que se recorrió.

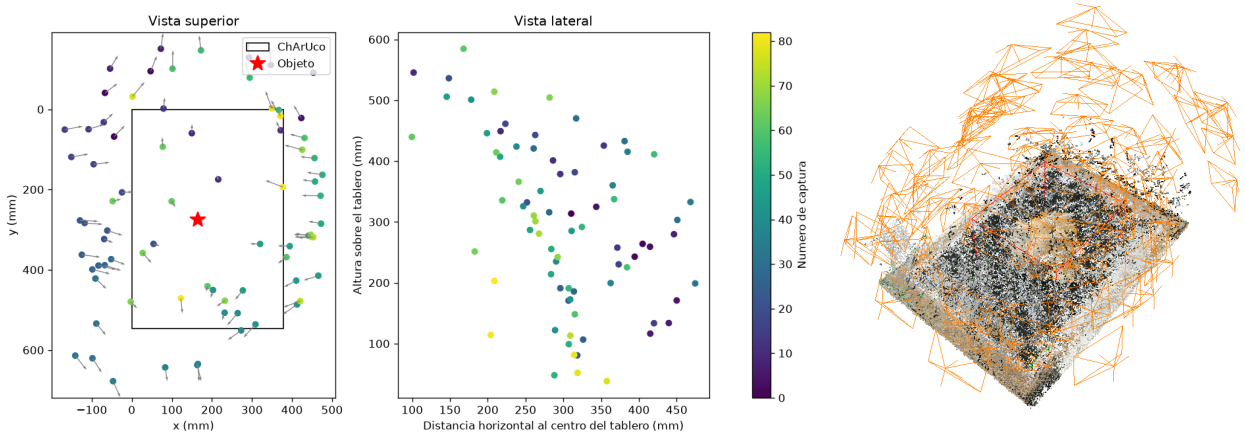


Figura 3: Poses de las 77 cámaras en el marco del ChArUco. Izquierda: centros (color = número de captura) y dirección del eje óptico. Derecha: frustums sobre la escena fusionada y la caja del objeto (rojo).

3.3. Fusión y optimización

La corrección por plano del tablero resultó de 0,55° y 3.9 mm en mediana (máximo de 19.4 mm). Esto es consistente con el desvío de escala de ±1,6 % entre vistas de la Tabla 2: a 500 mm, un 2 % equivale a 10 mm. El ICP posterior bajó el RMSE medio entre cada captura y la referencia de 1.37 a 1.18 mm (desplazamiento mediano de 4.2 mm), y aumentó un 41 % los puntos que pasan el filtro multivista (de 2.48 M a 3.50 M), lo que indica un mejor empalme.

Etapa	Puntos	% del anterior
Escena completa (77 nubes, sin optimizar)	104 748 216	–
Caja del objeto (`OrientedBoundingBox`)	5 435 170	5.2 %
ICP + consistencia multivista (≥20 vistas, voxel de 3 mm)	3 495 281	64.3 %
`voxel_down_sample` (1 mm)	259 442	7.4 %
`remove_statistical_outlier` (30, 2σ)	247 529	95.4 %
Cluster DBSCAN principal (**final**)	**243 961**	98.6 %

Cuadro 3: Puntos sobrevivientes en cada etapa. El `.ply` final binario (posición, color y normales) pesa 12.4 MB, mientras que la escena completa ocuparía ∼2.8 GB (posición en `double` + color).

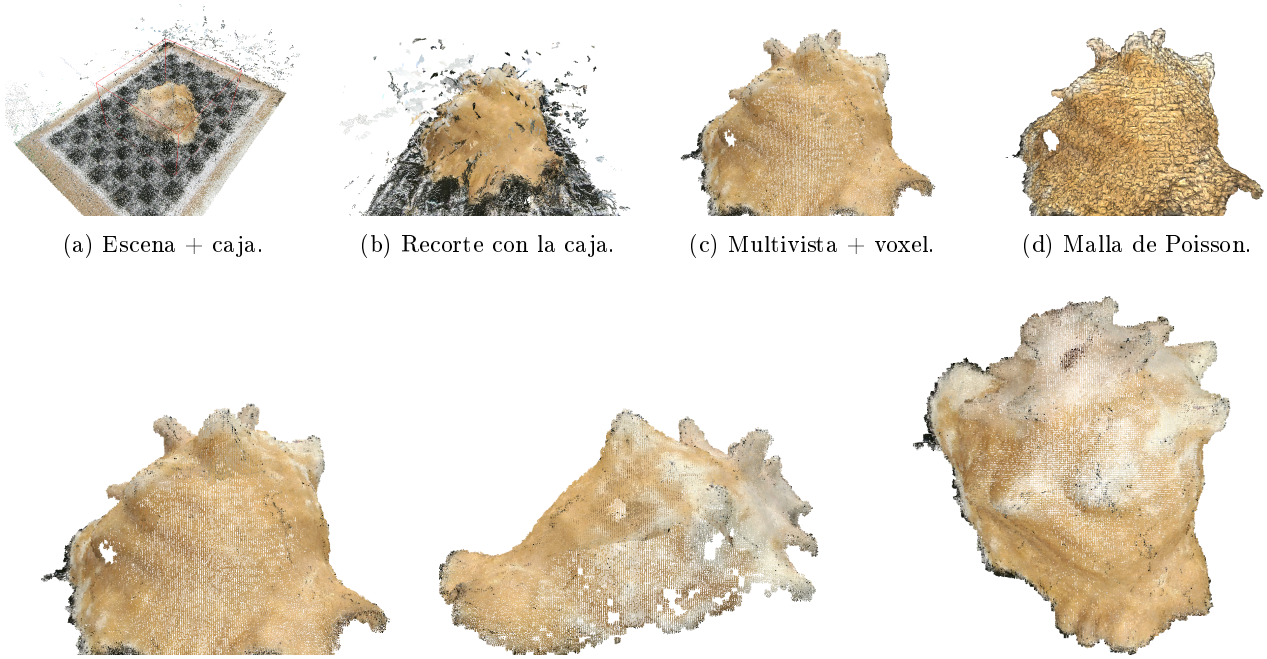
La Figura 4(b) muestra el principal problema después del recorte: una "pollera" de puntos blancos y negros del tablero que sube por los costados del caracol, además de fragmentos flotantes. Son *flying pixels*, píxeles en bordes de profundidad a los que SGBM asigna una disparidad intermedia entre el objeto y el fondo. El filtrado estadístico solo no alcanzaba a quitarlos, porque son densos localmente. El filtro multivista sí lo logra, porque cada vista genera la pollera en un lugar distinto (c). El submuestreo a 1 mm deja la nube en un 7.4 % y conserva las espinas y el labio del caracol.

3.4. Medición métrica

	Rectángulo mínimo + plano	OBB mínima Open3D	Rango en 3 corridas	Real (regla)	Error relativo
Alto	**121.8**	116.2	121.7–121.8	?	?
Largo	**219.3**	221.2	216.6–219.3	?	?
Ancho	**202.0**	209.6	201.7–202.2	?	?

Cuadro 4: Dimensiones estimadas del caracol, en mm.

La Tabla 4 resume la medición. Usamos como resultado el método que aprovecha el plano del tablero (Figura 5), porque el alto queda definido respecto de la superficie de apoyo real. La OBB mínima de Open3D no restringe la



(a) Escena + caja. (b) Recorte con la caja. (c) Multivista + voxel. (d) Malla de Poisson.

Figura 4: Arriba: etapas de la optimización. Abajo: nube final optimizada desde el frente, de costado y desde arriba.

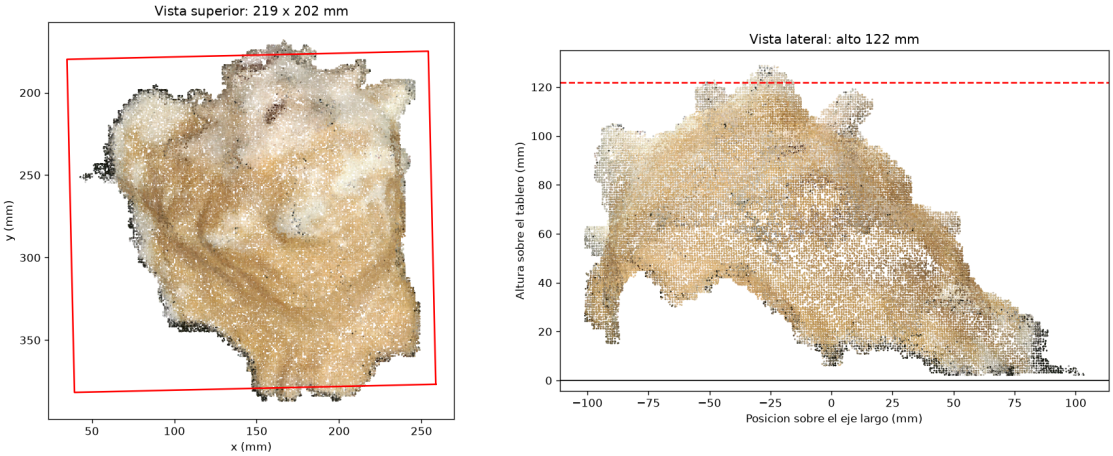


Figura 5: Medición sobre la nube final: rectángulo de área mínima de la proyección sobre el tablero (izquierda) y perfil a lo largo del eje largo, con el plano $z = 0$ y el alto (derecha).

orientación y se inclina para abrazar el objeto, por lo que mezcla alto con ancho (116 vs. 122 mm). Ejecutamos el pipeline completo tres veces: como el RANSAC de la corrección por plano no es determinista, las medidas varían en un rango de hasta 2.7 mm (1.2 %), una cota de la repetibilidad del método. El sesgo medio de escala de la calibración validada es del 0.02 %, y el desvío por vista ($\pm$1,6 %) se promedia entre las 77 capturas y queda compensado por ICP, así que la escala no es la fuente dominante: pesan más la definición de los bordes (la base del labio, que toca el tablero, y la punta de las espinas) y el ruido residual en el contorno.

3.5. Discusión

Problemas encontrados. *(1)* Los *flying pixels* de SGBM fueron el principal obstáculo para tener una nube limpia, y los resolvimos con el filtro de consistencia multivista. *(2)* Hay una inconsistencia de escala de $\pm$1.6 % por vista entre la triangulación y la pose PnP. La atribuimos a que la profundidad depende de la baseline y de la disparidad, mientras que la pose depende de K y del tamaño del tablero, y los errores de calibración afectan a cada una de forma distinta. La compensamos con la corrección por plano y con ICP. *(3)* La calibración con checkerboard tenía un sesgo de escala del 1.85 % en las distancias de captura, que corregimos con el refinamiento de $(R, \mathbf{t})$. Esto subraya que el RMS de reproyección por sí solo no garantiza precisión métrica. *(4)* En 6 capturas el tablero no fue detectable.

Limitaciones. El error de profundidad crece como Z^2: con $f = 600\,\text{px}$ y $B = 59\,\text{mm}$, un error de $0.5\,\text{px}$ en la disparidad equivale a $\sim$1.3 mm a 300 mm, pero a $\sim$4.3 mm a 550 mm. Las vistas lejanas aportan menos precisión. La base del objeto apoyada sobre el tablero nunca es visible, así que la nube no es cerrada y la malla de Poisson inventa esa cara. Las zonas sin textura (interior blanco de la abertura) quedan con menos densidad. La caja del objeto se eligió a mano, a partir de una fusión preliminar.

Mejoras posibles. Usar estéreo con redes neuronales (RAFT-Stereo, CREStereo), que da disparidades más completas y bordes más nítidos. Ajustar por *bundle adjustment* todas las poses junto con las esquinas del ChArUco. Fusionar volumétricamente (TSDF), que promedia la profundidad en lugar de acumular puntos. Capturar más cerca y a distancia constante (por ejemplo, con un plato giratorio). Calibrar a las mismas distancias de trabajo de la captura.